

2. Extraiga la llamada a la raíz cuadrada.

La primera optimización consistió en sacar la llamada a la función sqrt de la prueba de límite, por si acaso el compilador no la estuviera optimizando correctamente; esta es más rápida:

```
int es_primo(int n) {
    lim largo = (int) sqrt(n);
    para (int i=2; i<=lim; i++) si (n%i == 0) devolver 0; devolver 1;}
```

3. Reexpresión de la raíz cuadrada.

```
int es_primo(int n) {
    para (int i=2; i*i<=n; i++) si (n%i == 0) devolver 0; devolver 1;
}
```

4. No necesitamos comprobar los números pares.

```
int es_primo(int n) {
    si (n == 1) devuelve 0; si (n == 2)           // 1 NO es un número primo //
    devuelve 1; si (n % 2 == 0) devuelve         // 2 es un número primo
    0; para (int i = 3; i * i <= n; i += 2)       // Ningún número primo es par, excepto el 2 //
                                                // comienza desde el 3, salta 2 números // no es
                                                // necesario comprobar los números pares
    si (n%i == 0)
        devolver 0;
    devolver 1;
}
```

5. Otras propiedades principales

Un poco de reflexión debería indicarte que ningún número primo puede terminar en 0, 2, 4, 5, 6 u 8, quedando solo 1, 3, 7 y 9. Es rápido y sencillo. Memoriza esta técnica. Te será muy útil para tus tareas de programación que involucren números primos relativamente pequeños (enteros de 16 bits del 1 al 32767). Esta comprobación de divisibilidad (pasos 1 a 5) no será adecuada para números más grandes. Primer primo y único primo par: 2. El primo más grande en el rango de enteros de 32 bits: $2^{31} - 1 = 2.147.483.647$

6. Comprobación de divisibilidad utilizando primos más pequeños que sqrt(N):

En realidad, podemos mejorar la comprobación de divisibilidad para números más grandes. Una investigación posterior concluye que un número N es primo si y solo si ningún número primo menor que la raíz cuadrada de N puede dividir a N.

¿Cómo se hace esto?

1. Crea una matriz grande. ¿De qué tamaño?
2. Supongamos que el número máximo de primos generados no será mayor que $2^{31}-1$ (2,147,483,647), un entero máximo de 32 bits.
3. Dado que necesitas primos más pequeños que $\sqrt{N}$, solo necesitas

Almacena números primos desde 1 hasta $\sqrt{2^{31}}$.

4. Un cálculo rápido mostrará que $\sqrt{2^{31}} = 46340.95$.
5. Tras realizar algunos cálculos, descubrirás que existen como máximo 4792 números primos en el rango de 1 a 46340,95. Por lo tanto, solo necesitas un array de aproximadamente 4800 elementos.
6. Genera esos números primos del 1 al 46340,955. Esto llevará tiempo, pero una vez que tengas esos 4792 números primos, podrás utilizarlos para determinar si un número mayor es primo o no.
7. Ahora tienes 4792 primeros números primos. Lo único que tienes que hacer a continuación es comprobar si un número grande N es primo o no dividiéndolo entre primos pequeños hasta la raíz cuadrada de N . Si encuentras al menos un primo pequeño que divida a N , entonces N no es primo; de lo contrario, N es primo.

Pequeña prueba de Fermat:

Este es un algoritmo probabilístico, por lo que no se puede garantizar la posibilidad de obtener la respuesta correcta. En un rango tan amplio como 1-1.000.000, la Prueba Pequeña de Fermat solo puede ser engañada por 255 números de Carmichael (para ver los números de Carmichael mencionados anteriormente, consulte la lista de números que pueden engañar a la Prueba Pequeña de Fermat). Sin embargo, se pueden realizar múltiples comprobaciones aleatorias para aumentar esta probabilidad.

Algoritmo de Fermat

Si $2^N \bmod N = 2$, entonces N tiene una alta probabilidad de ser un número primo.

Ejemplo:

Sea $N=3$ (sabemos que 3 es un número primo).

$2^3 \bmod 3 = 8 \bmod 3 = 2$, entonces N tiene una alta probabilidad de ser un número primo... y de hecho, realmente es primo.

Otro ejemplo:

Sea $N=11$ (sabemos que 11 es un número primo).

$2^{11} \bmod 11 = 2048 \bmod 11 = 2$, entonces N tiene una alta probabilidad de ser un número primo... de nuevo, esto también es realmente primo.

El tamiz de Eratóstenes:

El algoritmo de criba es el mejor para generar números primos. Genera una lista de primos muy rápidamente, pero requiere mucha memoria. Para ello, puedes usar indicadores booleanos (un valor booleano ocupa solo 1 byte).

Algoritmo para la criba de Eratóstenes para encontrar los números primos dentro de un rango L,U (inclusive), donde debe cumplirse $L \leq U$.

```
void tamiz(int L,int U) {
    entero i,j,d;
    d=U-L+1; /* En el rango de L a U, tenemos d=U-L+1 números. */ /* Usamos flag[i] para
    marcar si (L+i) es un número primo o no. */

    bool *flag=new bool[d];
    para (i=0;i<d;i++) flag[i]=true; /* por defecto: marcar todos como verdaderos */

    para (i=(L%2!=0);i<d;i+=2) flag[i]=false;

    /* criba por factores primos comenzando desde 3 hasta sqrt(U) */ para
    (i=3;i<=sqrt(U);i+=2) {
        si (i>L && !flag[iL]) continuar;

        /* elige el primer número a tamizar -- >=L,
        divisible por i, y no por i mismo! */ j=L/i*i; si (j<L)
        j+=i;
        if (j==i) j+=i; /* si j es un número primo, hay que empezar desde el siguiente */

        j-=L; /* cambia j al índice que representa j */ for (;j<d;j+=i)
        flag[j]=false;
    }

    si (L<=1) flag[1-L]=false; si (L<=2)
    flag[2-L]=true;

    para (i=0;i<d;i++) si (flag[i]) cout << (L+i) << " "; cout << endl;
}
```

CAPÍTULO 8 CLASIFICACIÓN

Definición de un problema de ordenación

Aporte: Una secuencia de N números $(a_1, a_2, \dots, a_N)$

Producción: Una permutación $(a_{p_1}, a_{p_2}, \dots, a_{p_N})$ de la secuencia de entrada tal que $a_{p_1} \leq a_{p_2} \leq \dots \leq a_{p_N}$

Aspectos a tener en cuenta en la clasificación

Estas son las dificultades de clasificación que también pueden darse en la vida real:

- A. Tamaño La principal preocupación es el tamaño de la lista a ordenar. A veces, la memoria del ordenador no es suficiente para almacenar todos los datos. Es posible que solo puedas guardar una parte de los datos en el ordenador en un momento dado; el resto probablemente tendrá que permanecer en un disco o cinta. Esto se conoce como el problema de la ordenación externa. Sin embargo, ten la seguridad de que el tamaño de los problemas en casi todos los concursos de programación nunca será tan grande como para que necesites acceder a un disco o cinta para realizar una ordenación externa (este acceso al hardware suele estar prohibido durante los concursos).
- B. Otro problema es el estabilidad del método de ordenación. Ejemplo: supongamos que usted es una aerolínea. Tiene una lista de los pasajeros de los vuelos del día. A cada pasajero se le asocia el número de su vuelo. Probablemente querrá ordenar la lista alfabéticamente. No hay problema... Luego, querrá reordenar la lista por número de vuelo para obtener listas de pasajeros para cada vuelo. De nuevo, "no hay problema"... - excepto que sería muy conveniente que, para cada lista de vuelo, los nombres siguieran estando en orden alfabético. Este es el problema de la ordenación estable.
- C. Para ser un poco más matemáticos, supongamos que tenemos una lista de elementos $\{x_i\}$ con x_a igual a x_b en lo que respecta a la comparación de ordenación y con x_a antes de x_b en la lista. El método de ordenación es estable si x_a seguro que vendrá antes que x_b en la lista ordenada.

Finalmente, tenemos el problema de clasificación de claves. Los elementos individuales que se van a ordenar pueden ser objetos muy grandes (por ejemplo, fichas de registro complejas). Todos los métodos de ordenación implican, naturalmente, mucho movimiento de los elementos que se ordenan. Si los elementos son muy grandes, esto puede consumir mucho tiempo de procesamiento, mucho más que el que se necesita simplemente para intercambiar dos números enteros en un arreglo.

Algoritmos de ordenación basados en comparaciones

Los algoritmos de ordenación basados en comparaciones implican la comparación entre dos objetos a y b para determinar una de las tres posibles relaciones entre ellos: menor que, igual o mayor.

Estos algoritmos de ordenación se ocupan de cómo utilizar esta comparación de forma eficaz, para minimizar la cantidad de comparaciones necesarias. Empecemos por la versión más sencilla y avancemos hasta los algoritmos de ordenación basados en comparaciones más sofisticados.

Ordenación de burbuja

Velocidad: $O(n^2)$, extremadamente lento

Espacio: El tamaño del array inicial

Complejidad de la codificación: Simple

Este es el algoritmo de ordenación más simple y (desafortunadamente) el peor. Este algoritmo realiza una doble pasada por el array e intercambia dos valores cuando es necesario.

Ordenación de burbuja (A)

```
para i <- length[A]-1 hasta 1
  para j <- 0 a i-1
    if (A[j] > A[j+1]) // cambia ">" por "<" para realizar una ordenación descendente
      temp <- A[j]
      A[j] <- A[j+1]
      A[j+1] <- temp
```

Ejecución a cámara lenta del algoritmo de ordenación de burbuja (**Atrevido**==región ordenada):

```
5 2 3 1 4
2 3 1 45
2 1 3 45
1 23 4 5
12 3 4 5
12 3 4 5>>hecho
```

PON A PRUEBA TUS CONOCIMIENTOS SOBRE LA ORDENACIÓN DE BURBUJAS

Resuelve problemas de UVa relacionados con el algoritmo de ordenación de burbuja:

[299 - Intercambio de trenes 612 -](#)

[Clasificación de ADN 10327 -](#)

[Ordenación por volteo](#)

Ordenación rápida

Velocidad: $O(n \log n)$, uno de los mejores algoritmos de ordenación.

Espacio: El tamaño del array inicial.

Complejidad de la codificación: Compleja, utilizando el enfoque de divide y vencerás.

El algoritmo de ordenación rápida es uno de los mejores conocidos. Utiliza el método de divide y vencerás y particiona cada subconjunto. Particionar un conjunto consiste en dividirlo en conjuntos disjuntos. Este algoritmo es mucho más rápido que el algoritmo de burbuja, conocido por su sencillez. Fue inventado por Car Hoare.

Clasificación rápida: idea básica

Particionar el arreglo en $O(n)$

Ordenar recursivamente el array de la izquierda en $O(\log^2 n)$ caso óptimo/promedio

Ordenar recursivamente el array de la derecha en $O(\log^2 n)$ caso óptimo/promedio

Pseudocódigo de ordenación rápida:

```
Ordenación rápida(A,p,r)
  si p < r
    q <- Partición(A,p,r)
    Ordenación rápida(A,p,q)
    Ordenación rápida(A,q+1,r)
```

Ordenación rápida para usuarios de C/C++

La biblioteca estándar de C/C++ `<stdlib.h>` contiene la función `qsort`.

Esta no es la mejor implementación de ordenación rápida del mundo, pero es lo suficientemente rápida y MUY FÁCIL de usar... por lo tanto, si estás usando C/C++ y necesitas ordenar algo, simplemente puedes llamar a esta función integrada:

```
qsort(<arrayname>,<size>,sizeof(<elementsize>),compare_function);
```

Lo único que necesitas implementar es la función `compare_function`, que recibe dos argumentos de tipo "const void", que se pueden convertir a la estructura de datos apropiada, y luego devuelve uno de estos tres valores:

negativo, si a debe estar antes que b

0, si a es igual a b

positivo, si a debería estar después de b

1. Comparación de una lista de números enteros

simplemente convierte a y b a números enteros.

Si $x < y$, xy es negativo; si $x == y$, $xy = 0$; si $x > y$, xy es positivo. xy es una forma abreviada de hacerlo :)

Invertir $*x - *y$ a $*y - *x$ para ordenar en orden descendente

```
int compare_function(const void *a,const void *b) {
    int *x = (int *) a; int *y =
    (int *) b; return *x - *y;
}
```

2. Comparar una lista de cadenas

Para comparando cadena, tú necesidad strcmp función adentro cadena.h biblioteca.

Por defecto, strcmp devolverá -ve, 0, ve según corresponda... Para ordenar en orden inverso, simplemente invierte el signo devuelto por strcmp.

```
# incluir <string.h>

int compare_function(const void *a,const void *b) {
    devolver (strcmp((char *)a,(char *)b));
}
```

3. Comparación de números de coma flotante

```
int compare_function(const void *a,const void *b) {
    doble *x = (doble *) a; doble *y =
    (doble *) b;
    // devuelve *x - *y; // esto es INCORRECTO... si (*x <
    *y) devuelve -1;
    else if (*x > *y) return 1; return 0;
}
```

4. Comparación de registros basada en una clave

A veces necesitas ordenar elementos más complejos, como registros. Aquí te mostramos la forma más sencilla de hacerlo usando la biblioteca qsort.

```
typedef struct {
    clave entera;
    valor doble;
} el_registro;
```

```
int compare_function(const void *a,const void *b) {  
    el_registro *x = (el_registro *) a; el_registro  
    *y = (el_registro *) b; return x->clave - y-  
    >clave;  
}
```

Clasificación en múltiples campos, técnica de clasificación avanzada

A veces, la ordenación no se basa en una sola clave.

Por ejemplo, ordenar la lista de cumpleaños. Primero se ordena por mes, luego, si hay empate en el mes, se ordena por fecha (obviamente), y finalmente por año.

Por ejemplo, tengo una lista de cumpleaños sin ordenar como esta:

```
24 - 05 - 1982 - Soleado 24 - 05 -  
1980 - Cecilia  
31 - 12 - 1999 - Fin del siglo XX 01 - 01 - 0001 - Inicio del  
calendario moderno
```

Tendré una lista ordenada como esta:

```
01 - 01 - 0001 - Inicio del calendario moderno 24 - 05 -  
1980 - Cecilia  
24 - 05 - 1982 - Soleado  
31-12-1999 - Finales del siglo XX
```

Para realizar una clasificación de múltiples campos como esta, tradicionalmente se elige una clasificación múltiple utilizando un algoritmo de clasificación que tenga la propiedad de "clasificación estable".

La mejor manera de ordenar campos múltiples es modificar la función `compare_function` de forma que se resuelvan los empates adecuadamente... Les daré un ejemplo usando una lista de cumpleaños nuevamente.

```
typedef struct {  
    entero día, mes, año;  
    carácter *nombre;  
} cumpleaños;  
  
int compare_function(const void *a,const void *b) {  
    cumpleaños *x = (cumpleaños *) a;  
    cumpleaños *y = (cumpleaños *) b;  
  
    Si (x->mes != y->mes) // meses diferentes  
        devolver x->mes - y->mes; // ordenar por mes
```



```

de lo contrario { // los meses son iguales..., prueba el segundo campo... día
    Si (x->día != y->día) // días diferentes

        return x->day - y->day; // ordenar por día else // días
        iguales, probar el tercer campo... año
        devolver x->año - y->año; // ordenar por año
    }
}

```

PON A PRUEBA TUS CONOCIMIENTOS SOBRE CLASIFICACIÓN EN MÚLTIPLES CAMPOS

10194 - Fútbol (también conocido como soccer)

Ordenación en tiempo lineal

a. El límite inferior de la ordenación basada en comparaciones es $O(n \log n)$.

Los algoritmos de ordenación que vemos arriba son de ordenación basada en comparación; utilizan funciones de comparación como $<$, $<=$, $=$, $>$, $>=$, etc. para comparar 2 elementos. Podemos modelar esta ordenación por comparación utilizando un modelo de árbol de decisión, y podemos demostrar que la altura más corta de este árbol es $O(n \log n)$.

b. Clasificación por conteo

Para el algoritmo Counting Sort, asumimos que los números están en el rango $[0..k]$, donde k es como máximo $O(n)$. Configuramos un arreglo de contador que cuenta cuántos duplicados hay en la entrada y reordenamos la salida en consecuencia, sin ninguna comparación. La complejidad es $O(n+k)$.

c. Ordenación por radix

Para el algoritmo Radix Sort, asumimos que la entrada es *Dakota del Norte* número de dígitos, donde d es razonablemente limitado.

El algoritmo Radix Sort ordenará estos números dígito por dígito, comenzando por el menos significativo hasta llegar al más significativo. Generalmente utiliza un algoritmo de ordenación estable para ordenar los dígitos, como el Counting Sort mencionado anteriormente.

Ejemplo:

aporte:

321

257

113
622

ordenar por tercer (último) dígito:

321
622
113
257

después de esta fase, El tercer (último) dígito está ordenado. .

ordenar por segundo dígito: 1

13
321
622
257

después de esta fase, El segundo y el tercer (último) dígito están ordenados. .

ordenar por segundo dígito:

113
257
321
622

después de esta fase, Todos los dígitos están ordenados. .

Para un conjunto de *Dakota del Norte* números de dígitos, lo haremos ~~d~~ paso de conteo de orden que tienen complejidad $O(n+k)$ Por lo tanto, la complejidad de Radix Sort es $O(d(n+k))$.

CAPÍTULO 9 BÚSQUEDA

La búsqueda es muy importante en informática. Está estrechamente relacionada con la ordenación. Normalmente buscamos después de ordenar los datos. ¿Recuerdas la búsqueda binaria? Este es el mejor ejemplo.

de un algoritmo que utiliza tanto la ordenación como la búsqueda.[2]

El algoritmo de búsqueda depende de la estructura de datos utilizada. Si queremos buscar en un grafo, entonces tenemos un conjunto de algoritmos bien conocidos como DFS, BFS, etc.

Búsqueda binaria

La aplicación más común de la búsqueda binaria es encontrar un valor específico en una lista ordenada. La búsqueda comienza examinando el valor en el centro de la lista; como los valores están ordenados, sabe si el valor aparece antes o después del valor central y busca en la mitad correspondiente de la misma manera. A continuación, se muestra un pseudocódigo sencillo que determina el índice de un valor dado en una lista ordenada `a` entre los índices `left` y `right`:

```
función binarySearch(a, valor, izquierda, derecha)
    si derecha < izquierda
        no se encontró el retorno
    medio := piso((izquierda+derecha)/2)
    si a[medio] = valor
        regresar a mitad de semana
    si valor < a[mid]
        BúsquedaBinaria(a, valor, izquierda, medio-1)
    demás
        BúsquedaBinaria(a, valor, medio+1, derecha)
```

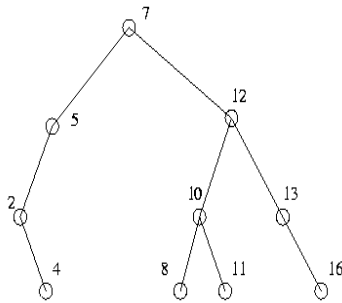
En ambos casos, el algoritmo termina porque en cada llamada recursiva o iteración, el rango de índices derecha menos izquierda siempre se hace más pequeño y, por lo tanto, eventualmente debe volverse negativo.

La búsqueda binaria es un algoritmo logarítmico que se ejecuta en tiempo $O(\log n)$. Específicamente, se necesitan $1 + \log_2 N$ iteraciones para obtener una respuesta. Es considerablemente más rápido que una búsqueda lineal. Puede implementarse mediante recursión o iteración, como se muestra arriba, aunque en muchos lenguajes se expresa de forma más elegante mediante recursión.

Árbol de búsqueda binaria

Los árboles de búsqueda binaria (BST, por sus siglas en inglés) permiten buscar rápidamente en una colección de objetos (cada uno con un valor real o entero) para determinar si un valor dado existe en la colección.

Básicamente, un árbol de búsqueda binaria es un árbol binario ordenado con raíz y ponderación de nodos. Esta descripción implica que cada nodo puede no tener hijos, tener un hijo izquierdo, un hijo derecho o ambos. Además, cada nodo tiene un objeto asociado, cuyo peso corresponde al valor de dicho objeto.



El árbol de búsqueda binaria también tiene la **propiedad** que el hijo izquierdo de cada nodo y los descendientes de su hijo izquierdo tengan un valor menor que el del nodo, y el hijo derecho de cada nodo y sus descendientes tengan un valor mayor o igual que el suyo. Árbol de búsqueda binaria

Los nodos se representan generalmente como una estructura con cuatro campos: un puntero al hijo izquierdo del nodo, un puntero al hijo derecho, el peso del objeto almacenado en este nodo y un puntero al objeto en sí. En ocasiones, para facilitar el acceso, también se añade un puntero al nodo padre.

¿Por qué son útiles los árboles de búsqueda binaria?

Dado un conjunto de n objetos, un árbol de búsqueda binaria tarda solo $O(\text{altura})$ tiempo en encontrar un objeto, suponiendo que el árbol no esté realmente desequilibrado; $O(\text{altura})$ es $O(\log n)$. Además, a diferencia de simplemente mantener un arreglo ordenado, insertar y eliminar objetos también solo toma $O(\log n)$ tiempo. También se pueden obtener todas las claves en un árbol de búsqueda binaria en orden recorriéndolo mediante un recorrido en orden $O(n)$.

Variaciones sobre árboles binarios

Existen varias variantes que aseguran que los árboles nunca son pobres. Los árboles splay, los árboles rojo-negro, los árboles B y los árboles AVL son algunos de los ejemplos más comunes. Todos ellos son mucho más complicados de programar, y los árboles aleatorios suelen ser buenos, por lo que generalmente no merece la pena utilizarlos.

Consejos: Si te preocupa que el árbol que has creado pueda ser defectuoso (por ejemplo, si se está creando insertando elementos desde un archivo de entrada), ordena los elementos aleatoriamente antes de insertarlos.

Diccionario

Un diccionario, o tabla hash, almacena datos con una forma muy rápida de realizar búsquedas. Digamos que hay una colección de objetos y una estructura de datos debe responder rápidamente a la pregunta: '¿Es

¿Este objeto se encuentra en la estructura de datos? (Por ejemplo, ¿esta palabra está en el diccionario?). Una tabla hash realiza esta tarea en menos tiempo que una búsqueda binaria.

La idea es la siguiente: encuentra una función que mapee los elementos de la colección a un entero entre 1 y x (donde x , en esta explicación, es mayor que el número de elementos en tu colección). Mantén un array indexado del 1 al x y almacena cada elemento en la posición que la función le asigne. Luego, para determinar si algo está en tu colección, simplemente introdúcelo en la función y comprueba si esa posición está vacía. Si no lo está, comprueba el elemento allí para ver si es el mismo que el que tienes en tu poder.

Por ejemplo, supongamos que la función está definida sobre palabras de 3 caracteres y es (primera letra + (segunda letra * 3) + (tercera letra * 7)) mod 11 (A=1, B=2, etc.), y las palabras son 'CAT', 'CAR' y 'COB'. Al usar ASCII, esta función toma 'CAT' y la asigna a 3, asigna 'CAR' a 0 y asigna 'COB' a 7, por lo que la tabla hash se vería así:

```
0: COCHE
1
2
3: GATO
4
5
6
7: COB
8
9
10
```

Ahora, para ver si 'BAT' está ahí, lo introducimos en la función hash para obtener 2. Esta posición en la tabla hash está vacía, por lo que no está en la colección. 'ACT', por otro lado, devuelve el valor 7, por lo que el programa debe comprobar si esa entrada, 'COB', es la misma que 'ACT' (no, por lo que 'ACT' tampoco está en el diccionario). Por otro lado, si la entrada de búsqueda es 'CAR', 'CAT', 'COB', el diccionario devolverá verdadero.

Manejo de colisiones

Esto pasa por alto un pequeño problema que surge. ¿Qué se puede hacer si dos entradas se corresponden con el mismo valor (por ejemplo, si queremos sumar 'ACT' y 'COB')? Esto se denomina colisión. Existen varias maneras de corregir las colisiones, pero este documento se centrará en un método llamado encadenamiento.

En lugar de tener una entrada en cada posición, mantenga una lista enlazada de entradas con el mismo valor hash. Por lo tanto, cada vez que se agrega un elemento, encuentre su posición y agréguelo a la lista.

comienzo (o final) de esa lista. Por lo tanto, para tener tanto 'ACT' como 'COB' en la tabla, se vería algo así:

```
0: COCHE
1
2
3: GATO
4
5
6
7: COB -> ACT
8
9
10
```

Para comprobar una entrada, es necesario examinar todos los elementos de la lista enlazada para determinar si el elemento no se encuentra en la colección. Esto, por supuesto, reduce la eficiencia del uso de la tabla hash, pero suele ser bastante útil.

Variaciones de la tabla hash

A menudo resulta muy útil almacenar más información que solo el valor. Por ejemplo, al buscar en un subconjunto pequeño de un subconjunto grande y usar la tabla hash para almacenar las ubicaciones visitadas, es posible que desee tener el valor de búsqueda de una ubicación en la tabla hash junto con ella.

CAPÍTULO 10 ALGORITMOS CODICIOSOS

Los algoritmos voraces son algoritmos que siguen la metaheurística de resolución de problemas de tomar la decisión óptima local en cada etapa con la esperanza de encontrar el óptimo global. Por ejemplo, al aplicar la estrategia voraz al problema del viajante de comercio se obtiene el siguiente algoritmo: "En cada etapa, visita la ciudad no visitada más cercana a la ciudad actual". [Enciclopedia Wiki]

Los algoritmos voraces no siempre encuentran la solución óptima global, ya que generalmente no procesan todos los datos de forma exhaustiva. Pueden tomar decisiones demasiado pronto, lo que les impide encontrar la mejor solución global posteriormente. Por ejemplo, todos los algoritmos voraces conocidos para el problema de coloración de grafos y otros problemas NP-completos no siempre encuentran soluciones óptimas. Sin embargo, son útiles porque se conciben rápidamente y suelen proporcionar buenas aproximaciones al óptimo.

Si se demuestra que un algoritmo voraz produce el óptimo global para una clase de problemas determinada, suele convertirse en el método preferido. Ejemplos de estos algoritmos voraces son el algoritmo de Kruskal y el de Prim para encontrar árboles de expansión mínima, así como el algoritmo para encontrar árboles de Huffman óptimos. La teoría de los matroides, junto con la teoría aún más general de los greedoides, proporciona clases completas de este tipo de algoritmos.

En general, los algoritmos voraces tienen cinco pilares:

- ◆ Un conjunto de candidatos, a partir del cual se crea una solución.
- ◆ Una función de selección que elige al mejor candidato para añadirlo a la solución.
- ◆ Una función de viabilidad, que se utiliza para determinar si un candidato puede utilizarse para contribuir a una solución.
- ◆ Una función objetivo, que asigna un valor a una solución, o a una solución parcial, y
- ◆ Una función de solución, que indicará cuándo hemos encontrado una solución completa.

Reparación de graneros

Hay una larga lista de puestos, algunos de los cuales deben cubrirse con tabloncillos. Puedes usar hasta N ($1 \leq N \leq 50$) tabloncillos, cada uno de los cuales puede cubrir cualquier número de puestos consecutivos. Cubre todos los puestos necesarios, procurando cubrir el menor número posible de puestos en total. ¿Cómo lo resolverás?

La idea básica de los algoritmos voraces es construir soluciones grandes a partir de soluciones más pequeñas. Sin embargo, a diferencia de otros enfoques, los algoritmos voraces solo conservan la mejor solución que encuentran a medida que avanzan. Así, para el problema de ejemplo, para construir la solución para $N = 5$, encuentran la mejor solución para $N = 4$ y luego la modifican para obtener una solución para $N = 5$. Ninguna otra solución para $N = 4$ se considera en ningún momento.

Los algoritmos voraces son rápidos, generalmente lineales o cuadráticos y requieren poca memoria adicional. Lamentablemente, no suelen ser correctas. Pero cuando funcionan, a menudo son fáciles de implementar y lo suficientemente rápidas de ejecutar.

Problemas con los algoritmos voraces

Los algoritmos voraces presentan dos problemas básicos.

1. Cómo construir

¿Cómo se crean soluciones más grandes a partir de soluciones más pequeñas? En general, esto depende del problema. Para el problema de ejemplo, la forma más obvia de pasar de cuatro tableros a cinco es elegir un tablero y quitar una sección, creando así dos tableros a partir de uno. Se debe elegir quitar la sección más grande de cualquier tablero que cubra solo los puestos que no necesitan cubrirse (para minimizar el número total de puestos cubiertos).

Para retirar una sección de cubículos cubiertos, tome la tabla que abarca dichos cubículos y divídala en dos tablas: una que cubra los cubículos anteriores a la sección y otra que cubra los cubículos posteriores a la segunda.

2. ¿Funciona?

El verdadero desafío para el programador radica en que las soluciones voraces no siempre funcionan. Aunque parezcan funcionar para la entrada de muestra, la entrada aleatoria y todos los casos imaginables, si existe un caso en el que no funcionen, al menos uno (¡sí no más!) de los casos de prueba de los jueces serán de esa forma.

Para el problema de ejemplo, para comprobar que el algoritmo voraz descrito anteriormente funciona, considere lo siguiente:

Supongamos que la solución no incluye el gran espacio que el algoritmo eliminó, sino uno más pequeño. Al combinar los dos tableros al final del espacio más pequeño y dividir el tablero a lo largo del espacio más grande, se obtiene una solución que utiliza la misma cantidad de tableros que la solución original, pero que cubre menos puestos. Esta nueva solución es mejor, por lo que la suposición es errónea y siempre deberíamos optar por eliminar el espacio más grande.

Si la respuesta no contiene este hueco en particular, pero sí otro igual de grande, al realizar la misma transformación se obtiene una respuesta que utiliza la misma cantidad de tableros y cubre la misma cantidad de puestos que la otra respuesta. Esta nueva respuesta es igual de buena que la solución original, pero no mejor, por lo que podemos elegir cualquiera de las dos.

Por lo tanto, existe una respuesta óptima que contiene la brecha grande, de modo que en cada paso siempre hay una respuesta óptima que es un superconjunto del estado actual. Así, la respuesta final es óptima.

Si existe una solución voraz, úsala. Son fáciles de programar, fáciles de depurar, se ejecutan rápidamente y consumen poca memoria; básicamente, definen un buen algoritmo en términos de concursos. El único requisito que falta en esa lista es la corrección. Si el algoritmo voraz encuentra la respuesta correcta, úsala, pero no te dejes engañar pensando que la solución voraz funcionará para todos los problemas.

Ordenar una secuencia de tres valores

Se te proporciona una secuencia de tres valores (1, 2 o 3) de longitud hasta 1000. Encuentra el conjunto mínimo de intercambios para ordenar la secuencia.

La secuencia tiene tres partes: la parte que será 1 cuando esté ordenada, la parte 2 cuando esté ordenada y la parte 3 cuando esté ordenada. El algoritmo voraz intercambia tantos 1 de la parte 2 como sea posible con 2 de la parte 1, tantos 1 de la parte 3 como sea posible con 3 de la parte 1, y 2 de la parte 3 con 3 de la parte 2. Una vez que no queda ninguno de estos tipos, los elementos restantes fuera de lugar deben rotarse en una dirección u otra en grupos de 3. Puedes ordenarlos de forma óptima intercambiando todos los 1 a su lugar y luego todos los 2 a su lugar.

Análisis: Obviamente, un intercambio puede colocar como máximo dos elementos en su lugar, por lo que todos los intercambios del primer tipo son óptimos. Además, es evidente que utilizan diferentes tipos de elementos, por lo que no hay "interferencia" entre ellos. Esto significa que el orden no importa. Una vez realizados esos intercambios, lo mejor que se puede hacer son dos intercambios por cada tres elementos que no estén en la posición correcta, que es lo que se logrará en la segunda parte (por ejemplo, todos los 1 se colocan en su lugar, pero ningún otro; entonces solo quedan 2 en el lugar de los 3 y viceversa, que se pueden intercambiar).

Ordenación topológica

Dada una colección de objetos, junto con algunas restricciones de orden, como "A debe estar antes que B", encuentre un orden de los objetos tal que se cumplan todas las restricciones de orden.

Algoritmo: Crea un grafo dirigido sobre los objetos, donde exista un arco de A a B si "A debe estar antes que B". Recorre los objetos en un orden arbitrario. Cada vez que encuentres un objeto con grado de entrada 0, colócalo vorazmente al final del orden actual, elimina todos sus arcos de salida y repite la misma comprobación en sus (antiguos) hijos. Si este algoritmo recorre todos los objetos sin incluirlos todos en el orden, no existe ningún orden que satisfaga las restricciones.

PON A PRUEBA TUS CONOCIMIENTOS CODICIOSOS

Resuelve problemas de UVa relacionados que utilizan algoritmos voraces:

10020 - Cobertura mínima

10340 - Todo incluido

10440 - Carga del ferry (II)

CAPÍTULO 11 PROGRAMACIÓN DINÁMICA

La programación dinámica (abreviada como PD) es una técnica de programación que puede reducir drásticamente el tiempo de ejecución de algunos algoritmos (aunque no todos los problemas presentan características de PD), pasando de exponencial a polinomial. Muchos problemas del mundo real (y cada vez más) solo pueden resolverse en un tiempo razonable mediante PD.

Para poder utilizar DP, el problema original debe tener:

1. **Subestructura óptima** propiedad:

La solución óptima al problema contiene en sí misma soluciones óptimas a los subproblemas.

2. **Subproblemas superpuestos** propiedad

Sin querer, recalculamos el mismo problema dos o más veces.

Existen dos tipos de programación dinámica (PD): podemos construir soluciones de subproblemas de menor a mayor (de abajo hacia arriba) o podemos guardar los resultados de las soluciones de los subproblemas en una tabla (de arriba hacia abajo). + memorización).

Comencemos con un ejemplo de la técnica de Programación Dinámica (PD). Examinaremos la forma más simple de subproblemas superpuestos. ¿Recuerdan Fibonacci? Un problema popular que crea mucha redundancia si se utiliza la recursión estándar. $f_n = f_{n-1} + f_{n-2}$.

La solución de programación dinámica (PD) de Fibonacci descendente registrará cada cálculo de Fibonacci en una tabla, por lo que no tendrá que volver a calcular el valor cuando lo necesite; basta con una simple búsqueda en la tabla (memorización). En cambio, la solución de PD ascendente construirá la solución a partir de números más pequeños.

Ahora veamos la comparación entre la solución sin programación dinámica (Non-DP) y la solución con programación dinámica (DP) (tanto de abajo hacia arriba como de arriba hacia abajo), que se muestra en el código fuente en C a continuación, junto con los comentarios correspondientes.

```
#include <stdio.h>

# define MAX 20 // para probar con un número mayor, ajuste este valor

int memo[MAX]; // arreglo para almacenar los cálculos anteriores

// El cálculo más lento e innecesario se repite int Non_DP(int n) {
    si (n==1 || n==2)
        devolver 1;
```

```
demás
    devolver Non_DP(n-1) + Non_DP(n-2);
}

// DP de arriba hacia abajo
int DP_Top_Down(int n) {
    // caso base
    si (n == 1 || n == 2)
        devolver 1;

    // Devuelve inmediatamente el resultado calculado previamente si
    (memo[n] != 0)
        devolver memo[n];

    // De lo contrario, haga lo mismo que Non_DP memo[n] =
    DP_Top_Down(n-1) + DP_Top_Down(n-2); return memo[n];
}

// DP más rápido, de abajo hacia arriba, almacena los resultados anteriores en un
array int DP_Bottom_Up(int n) {
    memo[1] = memo[2] = 1; // valores predeterminados para el algoritmo DP

    // de 3 a n (ya sabemos que fib(1) y fib(2) = 1 para (int i=3; i<=n; i++))

    memo[i] = memo[i-1] + memo[i-2];

    devolver memo[n];
}

void main() {
    entero z;

    // este será el más lento
    para (z=1; z<MAX; z++) printf("%d-",Non_DP(z)); printf("\n\n");

    // esto será mucho más rápido que el primero para (z=0;
    z<MAX; z++) memo[z] = 0;
    para (z=1; z<MAX; z++) printf("%d-",DP_Top_Down(z)); printf("\n\n");

    /* normalmente esto será lo más rápido */ para (z=0;
    z<MAX; z++) memo[z] = 0;
    para (z=1; z<MAX; z++) printf("%d-",DP_Bottom_Up(z)); printf("\n\n");
}
```

Multiplicación de Cadenas de Matrices (MCM)

Comencemos analizando el costo de multiplicar 2 matrices:

```

Multiplicación de matrices (A,B):
Si columnas[A] != columnas[B] entonces
    error "dimensiones incompatibles" en caso
    contrario
para i = 1 hasta filas[A] hacer
    para j = 1 hasta columnas[B] hacer
        C[i,j]=0
        para k = 1 hasta columnas[A] hacer
            C[i,j] = C[i,j] + A[i,k] * B[k,j]
devuelve C
  
```

Complejidad temporal = $O(pqr)$ donde $|A| = pxq$ y $|B| = qxr$

$|A| = 2 * 3$, $|B| = 3 * 1$, por lo tanto, para multiplicar estas 2 matrices, necesitamos $O(2*3*1)=O(6)$.

multiplicación escalar El resultado es la matriz C con $|C| = 2 * 1$

PON A PRUEBA TUS CONOCIMIENTOS SOBRE LA MULTIPLICACIÓN DE MATRICES

Resuelve problemas de UVa relacionados con la multiplicación de matrices:

[442 - Multiplicación de cadenas de matrices - Problema sencillo](#)

Problema de multiplicación de cadenas de matrices

Aporte : Matrices $A_1, A_2, \dots, A_{norte}$, cada A_i de tamaño $P_{i-1} \times P_i$

Producción : Producto A completamente entre paréntesis $A_2 \dots A_{norte}$ es minimiza el número de multiplicaciones escalares

Un producto de matrices se muestra completamente entre paréntesis si es una matriz única.

2. el producto de 2 productos matriciales completamente entre paréntesis

Ejemplo de problema MCM:

Tenemos 3 matrices y el tamaño de cada matriz

es: $A_1(10 \times 100)$, $A_2(100 \times 5)$, $A_3(5 \times 50)$

Podemos ponerlos entre paréntesis de dos maneras:

1. $(A_1(A_2A_3)) = 100 \times 5 \times 50 + 10 \times 100 \times 50 = 75000$
2. $((A_1A_2)A_3) = 10 \times 100 \times 5 + 10 \times 5 \times 50 = 7500$ (10 veces mejor)

Observe cómo el costo de multiplicar estas tres matrices difiere significativamente. El costo depende realmente de la elección de la paréntesis completa de las matrices. Sin embargo, verificar exhaustivamente todas las posibles paréntesis lleva un tiempo exponencial.

Ahora veamos cómo se puede resolver el problema MCM utilizando DP.

Paso 1: caracterizar la subestructura óptima de este problema.

Dejar $A_{yo..j}$ ($i < j$) denota el resultado de multiplicar $A_iA_{i+1}..A_j$.
 $A_{yo..j}$ se puede obtener dividiéndolo en $A_{yo..k}$ y $A_{k+1..j}$ y luego multiplicando los subproductos. Hay $j-i$ posibles descomposiciones (es decir, $k=i, \dots, j-1$).

Dentro del paréntesis óptimo de $A_{yo..j}$: (a) la paréntesis de $A_{yo..k}$ debe ser óptimo
 b) la inclusión entre paréntesis de $A_{k+1..j}$ debe ser óptimo

Porque si no son óptimas, entonces existen otras divisiones mejores, y deberíamos elegir esa división y no esta.

Paso 2: Formulación recursiva

Necesito encontrar $un_{1..n}$

Sea $m[i,j]$ = número mínimo de multiplicaciones escalares necesarias para calcular $A_{yo..j}$

Desde $A_{yo..j}$ se puede obtener rompiéndolo en $A_{yo..k}A_{k+1..j}$, tenemos

$$m[i,j] = 0, \text{ si } i=j$$

$$= \min_{i \leq k < j} \{ m[i,k] + m[k+1,j] + p_{i-1}p_kp_j \}, \text{ si } i < j$$

Sea $s[i,j]$ el valor k donde ocurre la división óptima.

Paso 3: Cálculo de los costos óptimos

Orden de cadena matricial(p)
 $n = \text{longitud}[p]-1$

```

para i = 1 a n hacer
  m[i,i] = 0
para l = 2 a n hacer
  para i = 1 hasta n-l+1 hacer
    j = i+l-1
    m[i,j] = infinito
    para k = i a j-1 hacer
      q = m[i,k] + m[k+1,j] + pi-1*pk*pj si q < m[i,j]
      entonces
        m[i,j] = q
        s[i,j] = k
devolver m y s

```

Paso 4: Construir una solución óptima

```

Imprimir-MCM(s,i,j)
si i=j entonces
  imprimir Ai
demás
  imprimir "(" + Imprimir-MCM(s,1,s[i,j]) + "*" + Imprimir-MCM(s,s[i,j]+1,j) +
  ")"

```

Nota Al igual que cualquier otra solución de programación dinámica, el problema de la matriz de coocurrencia (MCM) también se puede resolver mediante un algoritmo recursivo descendente con memorización. En ocasiones, si no se visualiza el enfoque ascendente, basta con modificar la solución recursiva descendente original incluyendo la memorización. Esto permite ahorrar mucho tiempo al evitar el cálculo repetitivo de subproblemas.

PON A PRUEBA TUS CONOCIMIENTOS SOBRE LA MULTIPLICACIÓN DE MATRICES EN CADENA

Resuelve problemas de UVA relacionados con la multiplicación de cadenas de matrices:

348 - Secuencia óptima de multiplicación de matrices - Utilice el algoritmo anterior

Subsecuencia común más larga (LCS)

Aporte : Dos secuencias

Producción : Una subsecuencia común más larga de esas dos secuencias, ver detalles a continuación.

Una secuencia Z es una **subsecuencia** de X $\langle x_1, \text{incógnita}_2, \dots, \text{incógnita}_{\text{metro}} \rangle$, si existe una sucesión estrictamente creciente $\langle i_1, i_2, \dots, i_k \rangle$ de índices de X tales que para todo $j=1, 2, \dots, k$, tenemos $x_{i_j} = z_j$.
ejemplo: X = $\langle B, C, A, D \rangle$ y Z = $\langle C, A \rangle$.

Una secuencia Z se llama **subsecuencia común** de la secuencia X e Y si Z es una subsecuencia tanto de X como de Y.

subsecuencia común más larga (LCS) es simplemente la "subsecuencia común" más larga de dos secuencias.

Un método de fuerza bruta para encontrar la subsecuencia común más larga (LCS, por sus siglas en inglés), como enumerar todas las subsecuencias y encontrar la más larga, consume demasiado tiempo. Sin embargo, un informático ha encontrado una solución de programación dinámica para el problema de la LCS. Aquí solo presentaremos el código final, escrito en C y listo para usar. Cabe destacar que este código ha sido ligeramente modificado y utilizamos variables globales (sí, no se trata de programación orientada a objetos).

```
#incluir <stdio.h>
# incluir <string.h>
# define MAX 100

carácter X[MÁX],Y[MÁX];
entero i,j,m,n,c[MÁX][MÁX],b[MÁX][MÁX];

int LCSlength() {

    m=strlen(X);
    n=strlen(Y);

    para (i=1;i<=m;i++) c[i][0]=0; para
    (j=0;j<=n;j++) c[0][j]=0;

    para (i=1;i<=m;i++)
        para (j=1;j<=n;j++) {
            si (X[i-1]==Y[j-1]) {
                c[i][j]=c[i-1][j-1]+1;
                b[i][j]=1; /* desde el noroeste */
            }

            else if (c[i-1][j]>=c[i][j-1]) { c[i][j]=c[i-1][j];

                b[i][j]=2; /* desde el norte */
            }

            demás {
                c[i][j]=c[i][j-1];
                b[i][j]=3; /* desde el oeste */
            }
        }
    devolver c[m][n];
}
```



```
void printLCS(int i,int j) {
    si (i==0 || j==0) regresar;

    si (b[i][j]==1) {
        printLCS(i-1,j-1);
        printf("%c",X[i-1]);}
    de lo contrario si (b[i][j]==2)
        printLCS(i-1,j);
    demás
        printLCS(i,j-1);
}

void main() {
    mientras (1) {
        obtiene(X);
        if (feof(stdin)) break; /* presione ctrl+z para terminar */ gets(Y);

        printf("Longitud de LCS -> %d\n",LCSlength()); /* contar la longitud */
        printLCS(m,n); /* reconstruir LCS */
        printf("\n");
    }
}
```

PON A PRUEBA TUS CONOCIMIENTOS SOBRE LA SUBSECUENCIA COMÚN MÁS LARGA

Resuelve problemas de UVa relacionados con LCS:

[531 - Compromiso](#)

[10066 - Las Torres Gemelas](#)

[10100 - Partido más largo](#)

[10192 - Vacaciones](#)

[10405 - Subsecuencia común más larga](#)

Editar distancia

Aporte: Dado dos cadena, Costo para supresión, inserción, y reemplazar
Producción: Indica las acciones mínimas necesarias para transformar la primera cadena en la segunda.

El problema de la distancia de edición es algo similar al LCS. La solución de programación dinámica para este problema es muy útil en biología computacional, por ejemplo, para comparar ADN.

Sea $d(\text{cadena1}, \text{cadena2})$ la distancia entre estas 2 cadenas.

Se utiliza una matriz bidimensional, $m[0..|s1|, 0..|s2|]$, para almacenar los valores de distancia de edición, de modo que $m[i,j] = d(s1[1..i], s2[1..j])$.

```

m[0][0] = 0;
para (i=1; i<length(s1); i++) m[i][0] = i; para (j=1;
j<length(s2); j++) m[0][j] = j;

para (i=0; i<length(s1); i++)
  para (j=0; j<length(s2); j++) {
    val = (s1[i] == s2[j]) ? 0 : 1; m[i][j] = min( m[i-1]
    [j-1] + val,
                                min(m[i-1][j]+1, m[i][j-1]+1));
  }

```

Para generar el rastro, utilice otro array para almacenar nuestra acción a lo largo del proceso. Analice estos valores posteriormente.

PON A PRUEBA TUS CONOCIMIENTOS SOBRE DISTANCIA DE EDICIÓN

164 - Computadora de cadenas

526 - Distancia de cadena y proceso de edición

Subsecuencia ascendente/descendente más larga (LIS/LDS)

Aporte : Dada una secuencia

Producción : La subsecuencia más larga de la secuencia dada tal que todos los valores en esta subsecuencia más larga son estrictamente crecientes/decrecientes.

Solución $O(N^2)$ DP para el problema LIS (este código comprueba los valores crecientes):

```

para i = 1 hasta total-1
  para j = i+1 hasta el total
    si altura[j] > altura[i] entonces
      Si length[i] + 1 > length[j] entonces
        longitud[j] = longitud[i] + 1
        predecesor[j] = i

```

Ejemplo de LIS

Secuencia de alturas: 1,6,2,3,5

Longitud inicial: [1,1,1,1,1] - porque la longitud máxima es al menos 1 rite... Predecesor

inicial: [nil,nil,nil,nil,nil] - se asume que no hay predecesor hasta ahora

Después del primer bucle de j:
 longitud: [1,2,2,2,2], porque 6,2,3,5 son todos > 1 predecesor:
 [nulo,1,1,1,1]
 Después del segundo bucle de j: (Sin cambios)
 longitud: [1,2,2,2,2], porque 2,3,5 son todos < 6 predecesor:
 [nil,1,1,1,1]
 Después de la tercera vuelta:
 longitud: [1,2,2,3,3], porque 3,5 son todos > 2 predecesor:
 [nil,1,1,3,3]

 Después de la cuarta vuelta:
 longitud: [1,2,2,3,4], porque 5 > 3 predecesor:
 [nil,1,1,3,4]

Podemos reconstruir la solución utilizando recursión y un array de predecesores.

¿Es $O(n^2)$ el mejor algoritmo para resolver LIS/LDS?

Afortunadamente, la respuesta es "No".

Existe un algoritmo $O(n \log k)$ para calcular LIS (para LDS, esto es simplemente un LIS inverso), donde k es el tamaño del LIS real.

Este algoritmo utiliza un invariante, donde para cada subsecuencia más larga de longitud l , terminará con el valor $A[l]$. (Observe que al mantener este invariante, el arreglo A estará ordenado naturalmente). La inserción posterior (solo realizará n inserciones, un número a la vez) utilizará una búsqueda binaria para encontrar la posición apropiada en estamatriz ordenada A

0 1 2 3 4 5 6 7 8 a - 7,10, 9, 2, 3, 8, 8, 1

A -ii, i, i, i, i, i, i, i (número de iteración, $i = \text{infinito}$) A -i -7, i, i, i, i, i, i (1)

A -i -7,10, i, i, i, i, i, i (2) A -i -7, 9, i, i, i, i, i, i (3)

A -i -7, 2, i, i, i, i, i, i (4) A -i -7, 2, 3, i, i, i, i, i (5)

A -i -7, 2, 3, 8, i, i, i, i (6) A -i -7, 2, 3, 8, i, i, i, i

(7) A -i -7, 1, 3, 8, i, i, i, i (8)

Como se puede observar, la longitud de LIS es 4, lo cual es correcto. Para reconstruir LIS, en cada paso, se almacena el array predecesor como en LIS estándar, pero esta vez se recuerdan los valores reales, ya que el array A solo almacena el último elemento de la subsecuencia, no los valores reales.

PON A PRUEBA TU CONOCIMIENTO SOBRE LA SUBSECUENCIA INCREMENTAL/DECREMENTAL MÁS LARGA

111 - Calificación de Historia 231 -
Probando al ATRAPADOR
481 - Lo que sube - se necesita $O(n \log k)$ LIS
497 - Iniciativa de Defensa Estratégica 10051 -
Torre de Cubos
10131 - Más grande es más inteligente

Mochila Zero-One

Aporte: N artículos, cada uno con varios V_i (Valor) y W_i (Peso) y tamaño máximo de mochila MW.

Producción: Valor máximo de los artículos que una persona puede llevar, si puede llevar o no llevar un artículo en particular.

Sea $C[i][w]$ el valor máximo si los elementos disponibles son $\{X_1, INCÓGNITA_2, \dots, INCÓGNITA_i\}$ y el tamaño de la mochila es w .

Si $i == 0$ o $w == 0$ (si no hay ningún artículo o la mochila está llena), no podemos tomar nada $C[i][w] = 0$
 Si $W_i > w$ (este artículo es demasiado pesado para nuestra mochila), omitimos este artículo $C[i][w] = C[i-1][w]$; si $W_i \leq w$, tomamos el máximo de "no tomar" o "tomar" $C[i][w] = \max(C[i-1][w], C[i-1][w-W_i]+V_i)$;

La solución se puede encontrar en $C[N][W]$;

```
para (i=0;i<=N;i++) C[i][0] = 0; para
(w=0;w<=MW;w++) C[0][w] = 0;

para (i=1;i<=N;i++)
  para (w=1;w<=MW;w++) {
    si (W[i] > w)
      C[i][w] = C[i-1][w];
    demás
      C[i][w] = max(C[i-1][w], C[i-1][w-W[i]]+V[i]);
  }
salida(C[N][MW]);
```

PON A PRUEBA TUS CONOCIMIENTOS SOBRE MOCHILAS DE 0 A 1 PUNTO

Resuelve problemas de UVa relacionados con el problema de la mochila 0-1:

10130 - Súper Venta

Contando el cambio

Aporte : Una lista de denominaciones y un valor N que se sustituirá por dichas denominaciones.

Producción : Número de maneras de cambiar N

Supongamos que tienes monedas de 1 centavo, 5 centavos y 10 centavos. Te piden que pagues 16 centavos; por lo tanto, debes dar una moneda de un centavo, una de cinco centavos y una de diez centavos. El algoritmo de cálculo del cambio se puede usar para determinar de cuántas maneras puedes pagar una cantidad de dinero.

El número de maneras de cambiar la cantidad A usando N tipos de monedas es igual a:

1. El número de maneras de cambiar la cantidad A usando todos los tipos de monedas excepto el primero, +
2. El número de maneras de cambiar la cantidad AD usando todos los N tipos de monedas, donde D es la denominación del primer tipo de moneda.

El proceso recursivo del árbol reducirá gradualmente el valor de A, y luego, utilizando esta regla, podremos determinar de cuántas maneras se pueden cambiar las monedas.

1. Si A es exactamente 0, debemos contarle como 1 forma de dar cambio.
2. Si A es menor que 0, debemos contarle como 0 formas de dar cambio.
3. Si N tipos de monedas es 0, debemos contar eso como 0 formas de dar cambio.

```
#incluir <stdio.h>
# define MAXTOTAL 10000 long long
nway[MAXTOTAL+1];

int moneda[5] = { 50,25,10,5,1 };

void main()
{
    entero i,j,n,v,c;
    scanf("%d",&n);
    v = 5;
    nway[0] = 1;
    para (i=0; i<v; i++) {
        c = moneda[i];
        para (j=c; j<=n; j++)
            nway[j] += nway[j-c];
    }
    printf("%lld\n",nway[n]);
}
```

PON A PRUEBA TUS CONOCIMIENTOS SOBRE EL CAMBIO

147 - Dólares

357 - Déjame contar las formas - Debe usar un número entero grande 674 - Cambio de moneda

Suma máxima del intervalo

Aporte : Una secuencia de números enteros

Producción : Una suma de un intervalo que comienza desde el índice i hasta el índice j (consecutivo), esta suma debe ser la máxima entre todas las sumas posibles.

Números: -16

Suma: -16

^

suma máxima

Números: 4-54 -3 4 4 -4 4-5

Suma: 4 -14 1 5 9 5 94

^

detener

^

suma máxima

Números:-2-3 -4

Suma:-2-3-4

^

suma máxima, pero negativa... (este es el máximo de todos modos)

Entonces, simplemente haga un barrido lineal de izquierda a derecha, acumule la suma elemento por elemento, inicie un nuevo intervalo cada vez que encuentre una suma parcial < 0 (y registre el mejor intervalo máximo actual encontrado hasta el momento)...

Al final, muestra el valor de los intervalos máximos.

PON A PRUEBA TUS CONOCIMIENTOS SOBRE LA SUMA MÁXIMA DE INTERVALOS

Resuelve problemas de UVa relacionados con la suma máxima de intervalos:

507 - Jill vuelve a la carga

Otros algoritmos de programación dinámica

Los problemas que se pueden resolver utilizando el algoritmo de Floyd-Warshall y sus variantes, pertenecientes a la categoría de algoritmos de búsqueda de la ruta más corta entre todos los pares de nodos, se pueden clasificar como soluciones de programación dinámica. Encontrará una explicación sobre el algoritmo de Floyd-Warshall en la sección de gráficos.

Aparte de eso, hay muchos problemas ad hoc que pueden utilizar la programación dinámica (PD); solo recuerde que cuando el problema que encuentre explota la subestructura óptima y los subproblemas repetitivos, aplique técnicas de PD, puede ser útil.

PON A PRUEBA TUS CONOCIMIENTOS DE DP

Resuelve problemas de UVa relacionados con la programación dinámica ad hoc:

[108 - Suma máxima 836 -](#)

[Submatriz más grande](#)

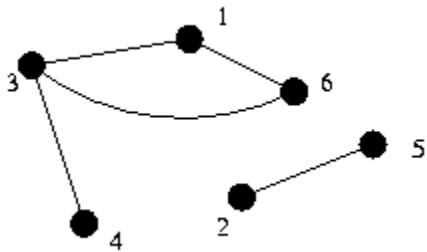
[10003 - Palitos de corte](#)

[10465 - Homer Simpson](#)

CAPÍTULO 12 GRÁFICOS

Agráfico es una colección de vértices V y una colección de bordes E consta de pares de vértices. Piensa en los vértices como "ubicaciones". El conjunto de vértices es el conjunto de todas las ubicaciones posibles. En esta analogía, las aristas representan caminos entre pares de esas ubicaciones. El conjunto E contiene todos los caminos entre las ubicaciones.[2]

Vértices y aristas



Normalmente, el gráfico se representa utilizando esa analogía. Los vértices son puntos o círculos, y las aristas son líneas que los conectan.

En este gráfico de ejemplo:

$V = \{1, 2, 3, 4, 5, 6\}$

$m_i = \{(1,3), (1,6), (2,5), (3,4), (3,6)\}$.

Cada vértice es un miembro del conjunto V . Un vértice es a veces llamado un nodo.

Cada borde es un miembro del conjunto E . Tenga en cuenta que algunos vértices podrían no ser el punto final de ninguna arista. Dichos vértices se denominan "aislado".

A veces, se asocian valores numéricos a las aristas, especificando longitudes o costos; estos grafos se denominan ponderado en los bordes gráficos (o ponderado gráficos). El valor asociado a una arista se llama peso del borde. Una definición similar se aplica a los grafos ponderados por nodos.

Telecomunicación

Dado un conjunto de ordenadores y un conjunto de cables que conectan pares de ordenadores, ¿cuál es el número mínimo de máquinas cuyo fallo provoca que dos de ellas no puedan comunicarse? (Las dos máquinas dadas no se bloquearán).

Gráfico Los vértices del grafo son las computadoras. Las aristas son los cables que las conectan. Problema de grafos: subgrafo dominante mínimo.

Montando las vallas

El granjero John posee una gran cantidad de cercas, las cuales debe revisar periódicamente para comprobar su integridad. Lleva un registro de sus cercas manteniendo una lista de los puntos donde se cruzan. Anota el nombre del punto y el nombre de una o dos cercas que lo tocan. Cada cerca tiene dos extremos, cada uno en un punto de intersección, aunque dicho punto de intersección puede coincidir con el extremo de una sola cerca.

Dado un trazado de cercas, calcula si existe una manera para que el granjero John pueda ir a caballo hasta todas sus cercas sin recorrerlas más de una vez. El granjero John puede empezar y terminar en cualquier punto, pero no puede cruzar sus campos (es decir, la única forma de desplazarse entre los puntos de intersección es siguiendo una cerca). Si existe una manera, encuentra una.

Gráfico El granjero John parte de los puntos de intersección y se desplaza entre ellos siguiendo las vallas. Por lo tanto, los vértices del grafo subyacente son los puntos de intersección y las vallas representan las aristas. Problema de grafos: Problema del viajante.

Movimientos del caballero

Dos casillas en un tablero de ajedrez de 8x8. Determina la secuencia más corta de movimientos de caballo de una casilla a la otra.

Gráfico El gráfico aquí es más difícil de visualizar. Cada ubicación en el tablero de ajedrez representa un vértice. Existe una arista entre dos posiciones si se trata de un movimiento válido de caballo. Problema de grafos: Camino más corto desde un único origen.

sobrevallado

El granjero John creó un enorme laberinto de vallas en un campo. Omitió dos segmentos de valla en los bordes, creando así dos «salidas» del laberinto. Es un laberinto «perfecto»; se puede encontrar una salida desde cualquier punto de su interior.

Dada la disposición del laberinto, calcule el número de pasos necesarios para salir del laberinto desde el punto "peor" del mismo (el punto que está "más lejos" de cualquiera de las salidas cuando se camina de forma óptima hacia la salida más cercana).

Aquí está qué uno particular $W=5$, $H=3$ laberinto aspecto como:

```

+ - + - + - +
- + | |
+ - + + - + +
| | | |
+ + - + - + +
| | |
+ - + + - + - +

```

Gráfico Los vértices del grafo son posiciones en la cuadrícula. Existe una arista entre dos vértices si representan posiciones adyacentes que no están separadas por una pared.

Terminología

Un borde es un bucle propio si tiene la forma (u,u) . El gráfico de ejemplo no contiene bucles propios.

Un gráfico es simple Si no contiene bucles propios ni una arista que se repita en E , un grafo se denomina multigrafo Si contiene una arista determinada más de una vez o contiene bucles. Para nuestros análisis, se asume que los grafos son simples. El grafo de ejemplo es un grafo simple.

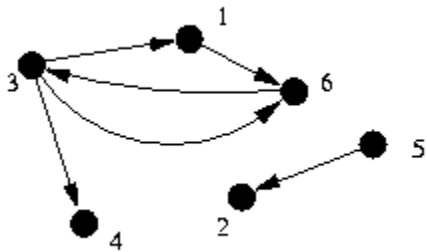
Una arista (u,v) es incidente al vértice u y al vértice v . Por ejemplo, la arista $(1,3)$ es incidente al vértice 3.

El grado El grado de un vértice es el número de aristas incidentes a él. Por ejemplo, el vértice 3 tiene grado 3, mientras que el vértice 4 tiene grado 1.

El vértice u es adyacente al vértice v si existe alguna arista a la que ambos inciden (es decir, hay una arista entre ellos). Por ejemplo, el vértice 2 es adyacente al vértice 5.

Se dice que un gráfico es escaso si el número total de aristas es pequeño comparado con el número total posible $((N \times (N-1))/2)$ y denso de lo contrario, para un grafo dado, no está bien definido si es denso o disperso.

Grafo dirigido



Los gráficos descritos hasta ahora se llaman no dirigido, ya que las aristas van en ambos sentidos. Hasta ahora, los grafos han indicado que si se puede viajar del vértice 1 al vértice 3, también se puede viajar del vértice 1 al vértice 3. En otras palabras, que $(1,3)$ esté en el conjunto de aristas implica que $(3,1)$ también lo está.

Sin embargo, a veces se necesita un gráfico dirigido, en cuyo caso los bordes tienen una dirección. En este

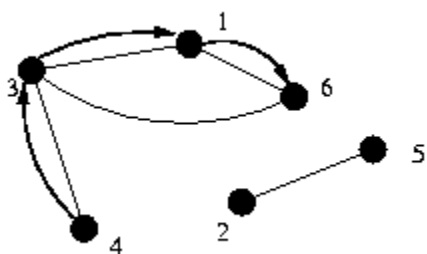
caso, los bordes se llaman arcos.

Los gráficos dirigidos se dibujan con flechas para indicar la dirección.

El grado de salida de un vértice es el número de arcos que comienzan en ese vértice. grado de ingreso El grado de entrada de un vértice es el número de arcos que terminan en ese vértice. Por ejemplo, el vértice 6 tiene un grado de entrada de 2 y un grado de salida de 1.

Se asume que un grafo es no dirigido a menos que se especifique lo contrario.

Caminos



Acamino Desde el vértice u hasta el vértice x es una secuencia de vértices $(v_0, v_1, \dots, v_k)$ tal que $v_0 = u$ y $v_k = x$ y (v_0, v_1) es una arista en el grafo, al igual que (v_1, v_2) , (v_2, v_3) , etc. La longitud de dicho camino es k .

Por ejemplo, en el grafo no dirigido anterior, $(4, 3, 1, 6)$ es un camino.

Se dice que este camino contener los vértices v_0, v_1 , etc., así como las aristas (v_0, v_1) , (v_1, v_2) , etc.

Se dice que el vértice x es accesible desde el vértice u si existe un camino desde u hasta x .

Un camino es simple si no contiene ningún vértice más de una vez.

Un camino es un ciclo si es un camino desde algún vértice a ese mismo vértice. Un ciclo es simple si no contiene ningún vértice más de una vez, excepto el vértice inicial (y final), que solo aparece como el primer y último vértice en el camino.

Estas definiciones se extienden de manera similar a los grafos dirigidos (por ejemplo, (v_0, v_1) , (v_1, v_2) , etc. deben ser arcos).

Representación gráfica

La elección de la representación de un gráfico es importante, ya que las diferentes representaciones tienen costes de tiempo y espacio muy diferentes.

Los vértices generalmente se identifican mediante numeración, de modo que se pueden indexar simplemente por su número. Por lo tanto, las representaciones se centran en cómo almacenar las aristas.

Lista de bordes

La forma más obvia de llevar un registro de las aristas es mantener una lista de los pares de vértices que representan las aristas en el grafo.

Esta representación es fácil de programar, relativamente fácil de depurar y bastante eficiente en cuanto al uso de espacio. Sin embargo, determinar las aristas incidentes a un vértice dado es costoso, al igual que determinar si dos vértices son adyacentes. Agregar una arista es rápido, pero eliminarla es difícil si se desconoce su ubicación en la lista.

Para grafos ponderados, esta representación también conserva un número adicional para cada arista: su peso. Extender esta estructura de datos para manejar grafos dirigidos es sencillo. Representar multigrafos también es trivial.

Ejemplo

	V1	V2
e1	4	3
e2	1	3
e3	2	5
e4	6	1
e5	3	6

Matriz de adyacencia

Una segunda forma de representar un gráfico utilizaba un *matriz de adyacencia*. Se trata de una matriz de $N \times N$ (donde N es el número de vértices). La entrada i, j contiene un 1 si la arista (i, j) pertenece al grafo; de lo contrario, contiene un 0. Para un grafo no dirigido, esta matriz es simétrica.

Esta representación es fácil de programar. Sin embargo, consume mucho menos espacio, especialmente en grafos grandes y dispersos. La depuración es más compleja, ya que la matriz es grande. Encontrar todas las aristas incidentes a un vértice dado es bastante costoso (lineal con respecto al número de vértices), pero comprobar si dos vértices son adyacentes es muy rápido. Agregar y eliminar aristas también son operaciones muy económicas.

En los grafos ponderados, el valor de la entrada (i, j) se utiliza para almacenar el peso de la arista. En un multigrafo no ponderado, la entrada (i, j) puede mantener el número de aristas entre los vértices. En un multigrafo ponderado, resulta más difícil extender esta funcionalidad.

Ejemplo

El grafo no dirigido de ejemplo estaría representado por la siguiente matriz de adyacencia:

	V1	V2	V3	V4	V5	V6
V1	0	0	1	0	0	1
V2	0	0	0	0	1	0
V3	1	0	0	1	0	1
V4	0	0	1	0	0	0
V5	0	1	0	0	0	0
V6	1	0	1	0	0	0

A veces resulta útil utilizar el hecho de que la entrada (i, j) de la matriz de adyacencia elevada a la k -ésima potencia da el número de caminos desde el vértice i al vértice j que constan de exactamente k aristas.

Lista de adyacencia

La tercera representación de una matriz consiste en registrar todas las aristas incidentes a un vértice dado. Esto se puede lograr utilizando un arreglo de longitud N , donde N es el número de vértices. La i -ésima entrada de este arreglo es una lista de las aristas incidentes al i -ésimo vértice (las aristas se representan mediante el índice del otro vértice incidente a dicha arista).

Esta representación es mucho más difícil de programar, especialmente si el número de aristas incidentes a cada vértice no está limitado, por lo que las listas deben ser enlazadas (o asignadas dinámicamente). Depurar esto es difícil, ya que seguir listas enlazadas es más complicado. Sin embargo, esta representación utiliza casi tanta memoria como la lista de aristas. Encontrar los vértices adyacentes a cada nodo es muy económico en esta estructura, pero comprobar si dos vértices son adyacentes requiere comprobar todas las aristas adyacentes a uno de los vértices. Agregar una arista es fácil, pero eliminar una arista es difícil si se desconocen las ubicaciones de la arista en las listas correspondientes.

Esta representación se puede extender para manejar grafos ponderados, manteniendo tanto el peso como el otro vértice incidente para cada arista, en lugar de solo el otro vértice incidente. Los multigrafos ya son representables. Los grafos dirigidos también se manejan fácilmente con esta representación, de varias maneras: almacenando solo las aristas en una dirección, manteniendo una lista separada de arcos entrantes y salientes, o indicando la dirección de cada arco en la lista.

Ejemplo

La representación de la lista de adyacencia del grafo no dirigido de ejemplo es la siguiente:

Vértice	Vértices adyacentes
1	3, 6
2	5
3	6, 4, 1
4	3
5	2
6	3, 1

Representación implícita

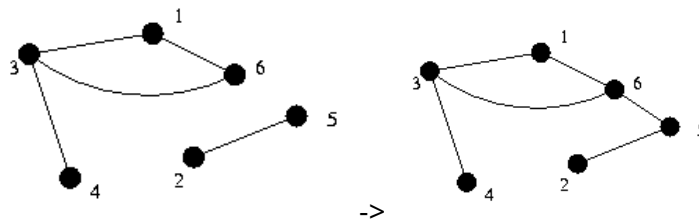
Para algunos grafos, no es necesario almacenar el grafo en sí. Por ejemplo, en los problemas de movimientos de caballo y sobreesgrima, es fácil calcular los vecinos de un vértice, comprobar la adyacencia y determinar todas las aristas sin almacenar realmente esa información; por lo tanto, no hay razón para almacenarla, ya que el grafo está implícito en los propios datos.

Si es posible almacenar el gráfico en este formato, generalmente es lo correcto, ya que ahorra mucho espacio de almacenamiento y reduce la complejidad del código, lo que facilita tanto su escritura como su depuración.

Si N es el número de vértices, M el número de aristas y $d_{\max}$ el grado máximo de un nodo, la siguiente tabla resume las diferencias entre las representaciones:

Eficiencia	Lista de bordes	Matriz ajustada	Lista de adyacentes
Espacio	$2 \cdot M$	N^2	$2 \cdot M$
Verificación de adyacencia	METRO	1	$d_{\max}$.
Lista de vértices adyacentes	METRO	norte	$d_{\max}$.
Agregar borde	1	1	1
Eliminar Edge	METRO	2	$2 \cdot d_{\max}$

Conexión



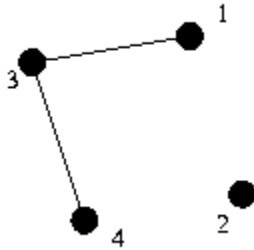
Se dice que un grafo no dirigido es conectado si existe un camino desde cada vértice a todos los demás vértices, el grafo de ejemplo no está conectado, ya que no hay un camino desde el vértice 2 al vértice 4. Sin embargo, si se añade una arista entre el vértice 5 y el vértice 6, el grafo se vuelve conectado.

Acomponente Un componente de un grafo es un subconjunto máximo de los vértices tal que cada vértice es alcanzable desde cualquier otro vértice del componente. El grafo del ejemplo original tiene dos componentes: $\{1, 3, 4, 6\}$ y $\{2, 5\}$. Nótese que $\{1, 3, 4\}$ no es un componente, ya que no es máximo.

Se dice que un grafo dirigido es fuertemente conectado si existe un camino desde cada vértice a todos los demás vértices.

Acomponente fuertemente conectado de un grafo dirigido es un vértice u y el conjunto de todos los vértices v tales que existe un camino de u a v y un camino de v a u .

Subgrafos

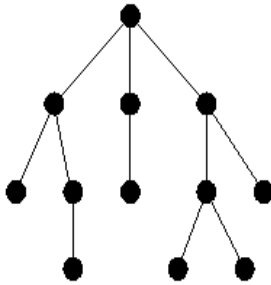


El grafo $G' = (V', E')$ es un subgrafo de $G = (V, E)$ si V' es un subconjunto de V y E' es un subconjunto de E .

El subgrafo de G inducido V' es el grafo (V', E') , donde E' consta de todas las aristas de E que están entre miembros de V' .

Por ejemplo, para $V' = \{1, 3, 4, 2\}$, el subgrafo es como el que se muestra ->

Árbol

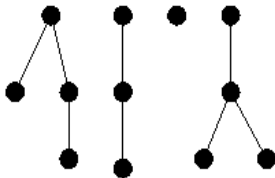


Se dice que un grafo no dirigido es un árbol si no contiene ciclos y está conectado.

Un árbol enraizado

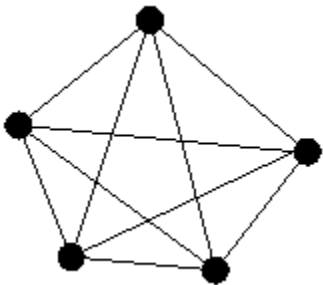
Muchos árboles son lo que se llama enraizado donde existe la noción del nodo "superior", que se denomina raíz. Por lo tanto, cada nodo tiene un padre, que es el nodo adyacente que está más cerca de la raíz, y puede tener cualquier número de hijos, que son el resto de los nodos adyacentes a él. El árbol de arriba se dibujó como un árbol con raíz.

Bosque



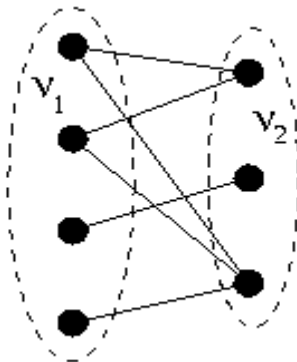
Un grafo no dirigido que no contiene ciclos se llama bosque. Un grafo acíclico dirigido se suele denominar grafo trozo de cuero.

Gráfico completo



Se dice que un gráfico es completo si hay una arista entre cada par de vértices.

Grafo bipartito



Se dice que un gráfico es bipartito si los vértices se pueden dividir en dos conjuntos V_1 y V_2 de tal manera que no haya aristas entre dos vértices de V_1 o dos vértices de V_2 .

Búsqueda no informada

La búsqueda es un proceso que consiste en considerar posibles secuencias de acciones; primero hay que formular un objetivo y luego utilizar ese objetivo para formular un problema.

La **problema** consta de cuatro partes: el **estado inicial**, un conjunto de **operadores**, la **prueba de objetivos** y la **función de costo**. El entorno del problema está representado por una

espacio de estados. Acaminos a través del espacio de estados desde el estado inicial hasta un estado objetivo es un **solución**.

En la vida real, la mayoría de los problemas están mal definidos, pero con cierto análisis, muchos problemas pueden encajar en el modelo de espacio de estados. Se puede utilizar un único algoritmo de búsqueda general para resolver cualquier problema; las variantes específicas del algoritmo incorporan diferentes estrategias. Los algoritmos de búsqueda se juzgan en función de **lo completo, optimización, complejidad temporal, y complejidad espacial**. La complejidad depende de b , el **factor de ramificación** en el espacio de estados, y d , el **profundidad** de la solución más superficial.

Los siguientes seis tipos de búsqueda (hay más, pero aquí solo mostramos seis) se clasifican como búsqueda no informada. Esto significa que la búsqueda no dispone de información sobre el número de pasos ni el coste del camino desde el estado actual hasta el objetivo; lo único que puede hacer es distinguir un estado objetivo de un estado que no lo es. La búsqueda no informada también se conoce a veces como búsqueda ciega.

Búsqueda en amplitud (BFS)

Búsqueda en amplitud Expande primero el nodo menos profundo del árbol de búsqueda. Es completo, óptimo para operadores de costo unitario y tiene una complejidad temporal y espacial de $O(b^d)$. Su complejidad espacial lo hace poco práctico en la mayoría de los casos.

Utilizando la estrategia BFS, primero se expande el nodo raíz, luego se expanden todos los nodos generados por el nodo raíz, y sus sucesores, y así sucesivamente. En general, todos los nodos en profundidad d en el árbol de búsqueda se expanden antes de los nodos en profundidad $d+1$.

Algorítmicamente:

```
BFS(G,s) {
  inicializar vértices;
  Q = {s};
  mientras (Q no esté vacío) {
    u = Desencolar(Q);
    para cada v adyacente a u hacer {
      si (color[v] == BLANCO) {
        color[v] = GRIS;
        d[v] = d[u]+1; // calcular d[] p[v] = u; //
        construir árbol BFS Enqueue(Q,v);
      }
    }
    color[u] = NEGRO;
  }
}
```

BFS se ejecuta en $O(V+E)$

Nota BFS puede calcular $d[v]$ = distancia del camino más corto de s a v , en términos del número mínimo de aristas de s a v (grafo no ponderado). Su árbol de búsqueda en anchura puede usarse para representar el camino más corto.

Solución BFS al problema popular de JAR

```
# incluir<stdio.h>
# incluir<conio.h>
# incluir<valores.h>

# define N 105
# define MAX MAXINT

int act[N][N], Q[N*20][3], cost[N][N]; int a, p, b, m, n, fin, na,
nb, front, rear;

void init()
{
    frente = -1, parte trasera = -1;
    para(int i=0; i<N; i++)
        para(int j=0; j<N; j++)
            costo[i][j] = MÁX;
    costo[0][0] = 0;
}

void nQ(int r, int c, int p) {
    Q[++trasera][0] = r, Q[trasera][1] = c, Q[trasera][2] = p;
}

void dQ(int *r, int *c, int *p) {
    *r = Q[++front][0], *c = Q[front][1], *p = front;
}

void op(int i)
{
    entero capital actualA, capital
    actualB; si(i==0)
        na = 0, nb = b;
    else if(i==1)
        nb = 0, na = a;
    else if(i==2)
        na = m, nb = b;
    else if(i==3)
        nb = n, na = a;
    else if(i==4)
    {
        si(!a && !b)
```

```

        devolver;
        currCapB = n - b;
        si(currCapB <= 0)
            devolver;
        si(a >= currCapB)
            nb = n, na = a, na -= currCapB;
        demás
            nb = b, nb += a, na = 0;
    }
    demás
    {
        si(!a && !b)
            devolver;
        currCapA = m - a;
        si(currCapA <= 0)
            devolver;
        si(b >= currCapA)
            na = m, nb = b, nb -= currCapA;
        demás
            nb = 0, na = a, na += b;
    }
}

void bfs()
{
    nQ(0, 0, -1);
    hacer{

        dQ(&a, &b, &p);
        si(a==fin)
            romper;
        para(int i=0; i<6; i++) {

            op(i); /* na, nb se modificará para esta función
                    según los valores de a, b
                    */
            si(costo[na][nb]>costo[a][b]+1) {

                costo[na][nb]=costo[a][b]+1; act[na]
                [nb] = i;
                nQ(na, nb, p);
            }
        }
    } mientras (trasero!=delantero);
}

void dfs(int p)
{
    int i = act[na][nb]; si(p== -1)

        devolver;
    na = Q[p][0], nb = Q[p][1]; dfs(Q[p]
    [2]);
    si(i==0)

```

```
        printf("Vacío A\n");
    else if(i==1)
        printf("B vacío\n");
    else if(i==2)
        printf("Rellenar A\n");
    else if(i==3)
        printf("Rellenar B\n");
    else if(i==4)
        printf("Vierta A en B\n");
    demás
        printf("Entrada B a A\n");
}

void main()
{
    clrscr();
    mientras(scanf("%d%d%d", &m, &n, &fin)!=EOF) {

        printf("\n");
        inicializar();
        bfs();
        dfs(Q[p][2]);
        printf("\n");
    }
}
```

Búsqueda de Coste Uniforme (UCS)

Búsqueda de coste uniformeExpande primero el nodo hoja de menor coste. Es un método completo y, a diferencia de la búsqueda en anchura, es óptimo incluso cuando los operadores tienen costes diferentes. Su complejidad espacial y temporal es la misma que la de la búsqueda en anchura.

BFS encuentra el estado objetivo menos profundo, pero esto no siempre es la solución de menor costo para una función de costo de ruta general. UCS modifica BFS expandiendo siempre el nodo de menor costo en la periferia.

Búsqueda en profundidad (DFS)

Búsqueda en profundidadEste método expande primero el nodo más profundo del árbol de búsqueda. No es ni completo ni óptimo, y tiene una complejidad temporal de $O(b^m)$ y una complejidad espacial de $O(bm)$, donde m es la profundidad máxima. En árboles de búsqueda de gran profundidad o de profundidad infinita, su complejidad temporal lo hace poco práctico.

El algoritmo DFS siempre expande uno de los nodos del nivel más profundo del árbol. Solo cuando la búsqueda llega a un callejón sin salida (un nodo que no es un objetivo y que no se puede expandir) retrocede y expande los nodos de niveles menos profundos.

Algorítmicamente:

```
DFS(G) {
  para cada vértice u en V
    color[u] = BLANCO;
  tiempo = 0; // variable global para
  cada vértice u en V

  si (color [u] == BLANCO)
    Visita DFS(u);
}

DFS_Visita(u) {
  color[u] = GRIS;
  tiempo = tiempo + 1; // variable global d[u] = tiempo; //
  calcular el tiempo de descubrimiento d[] para cada v
  adyacente a u
  si (color[v] == BLANCO) {
    p[v] = u; // construir árbol DFS
    DFS_Visita(v);
  }
  color[u] = NEGRO;
  tiempo = tiempo + 1; // variable global f[u] = tiempo; //
  calcular tiempo de finalización f[]
}
```

DFS se ejecuta en $O(V+E)$

DFS se puede utilizar para clasificar las aristas de G:

1. Bordes de árboles: bordes en el bosque en profundidad
2. Aristas de retroceso: aristas (u,v) que conectan un vértice u con un ancestro v en un árbol de búsqueda en profundidad.
3. Aristas hacia adelante: aristas no arbóreas (u,v) que conectan un vértice u con un descendiente v en un árbol en profundidad.
4. Bordes transversales: todos los demás bordes

Un grafo no dirigido es acíclico si y solo si una búsqueda en profundidad no produce aristas de retroceso.

Implementación del algoritmo DFS

Formar una cola de un solo elemento que consista en el nodo raíz.

Hasta que la cola esté vacía o se haya alcanzado el objetivo, determine si el primer elemento de la cola es el nodo objetivo. Si el primer elemento es el nodo objetivo, no haga nada. Si el primer elemento no es el nodo objetivo, elimine el primer elemento de la cola y agregue los hijos del primer elemento, si los hay, a la cola. **frente** de la cola.

Si se ha encontrado el nodo objetivo, anuncie el éxito; de lo contrario, anuncie el fracaso.

Nota: Esta implementación difiere de BFS en la inserción de los hijos del primer elemento, DFS desde **FRENTE** mientras que BFS de **ATRÁS**. El peor caso para DFS es el mejor caso para BFS y viceversa. Sin embargo, evite usar DFS cuando los árboles de búsqueda sean muy grandes o con profundidad máxima infinita.

Problema de N Queens

Coloca n reinas en un tablero de ajedrez de nxn de manera que ninguna reina sea atacada por otra reina.

La solución más obvia para el código es añadir reinas al tablero de forma recursiva, una por una, probando todas las posibles ubicaciones. Es fácil aprovechar el hecho de que debe haber exactamente una reina en cada columna: en cada paso de la recursión, basta con elegir dónde colocar la reina en la columna actual.

Solución DFS

```
1 búsqueda(col)
2   si se rellenan todas las columnas
3     Imprimir solución y salir para
4     cada fila
5     si board(row, col) no es atacado
6       colocar la reina en (fila, columna)
7       buscar(columna+1)
8       Eliminar la reina en (fila, columna)
```

Al llamar a search(0) se inicia la búsqueda. Esta se ejecuta rápidamente, ya que hay relativamente pocas opciones en cada paso: una vez que hay algunas reinas en el tablero, el número de casillas no atacadas disminuye drásticamente.

Este es un ejemplo de búsqueda en profundidad, ya que el algoritmo recorre el árbol de búsqueda lo más rápido posible: una vez colocadas k reinas en el tablero, se examinan tableros con aún más reinas antes de examinar otros posibles tableros con solo k reinas. Esto es aceptable, pero a veces es preferible encontrar las soluciones más sencillas antes de probar las más complejas.

La búsqueda en profundidad comprueba cada nodo de un árbol de búsqueda en busca de alguna propiedad. El árbol de búsqueda podría tener este aspecto:

La complejidad espacial de la búsqueda en profundidad iterativa es la misma que la de la búsqueda en profundidad: $O(n)$. La complejidad temporal, en cambio, es más compleja. Cada búsqueda en profundidad truncada que se detiene en la profundidad k requiere un tiempo de ck . Entonces, si d es el número máximo de decisiones, la búsqueda en profundidad iterativa requiere un tiempo de $c^0 + c^1 + c^2 + \dots + c^d$.

¿Cuál usar?

Una vez que hayas identificado un problema como un problema de búsqueda, es importante elegir el tipo de búsqueda adecuado. Aquí tienes algunas cosas que debes tener en cuenta.

Buscar	Tiempo	Espacio	Cuándo usar
DFS	$O(c \cdot k)$	De acuerdo)	De todas formas, debes buscar en el árbol, conocer el nivel en el que están las respuestas, o no estarás buscando el número más superficial.
BFS	$O(c^d)$	$O(c^d)$	Si conoces las respuestas que están muy cerca de la cima del árbol, o quieres la respuesta más superficial.
DFS+ID	$O(c^d)$	Sobredosis)	Quiero hacer BFS, no tengo suficiente espacio y puedo dedicarle tiempo.

d es la profundidad de la respuesta, k es la profundidad buscada, $d \leq k$.

Recuerda las propiedades de ordenación de cada búsqueda. Si el programa necesita generar una lista ordenada de la solución más corta a la más larga (en términos de distancia al nodo raíz), utiliza la búsqueda en amplitud o la búsqueda en profundidad iterativa. Para otros órdenes, la búsqueda en profundidad es la estrategia adecuada.

Si no hay tiempo suficiente para recorrer todo el árbol, utilice el algoritmo con mayor probabilidad de encontrar la respuesta. Si se espera que la respuesta se encuentre en una de las filas de nodos más cercanas a la raíz, utilice la búsqueda en anchura o la búsqueda en profundidad iterativa. Por el contrario, si se espera que la respuesta se encuentre en las hojas, utilice la búsqueda en profundidad, que es más sencilla.

Asegúrese de tener en cuenta las limitaciones de espacio. Si la memoria es insuficiente para mantener la cola de búsqueda en amplitud, pero hay tiempo disponible, utilice la búsqueda en profundidad iterativa.

Búsqueda con profundidad limitada

Búsqueda con profundidad limitada Establece un límite a la profundidad que puede alcanzar una búsqueda en profundidad. Si el límite coincide con la profundidad del estado objetivo menos profundo, se minimizan la complejidad temporal y espacial.

DLS deja de avanzar cuando la profundidad de búsqueda es mayor que la que hemos definido.

Búsqueda iterativa dependiente

Búsqueda de profundización iterativa Realiza una búsqueda con profundidad limitada y límites crecientes hasta encontrar un objetivo. Es completa y óptima, y tiene una complejidad temporal de $O(b^d)$.

IDS es una estrategia que evita el problema de elegir el límite de profundidad óptimo probando todos los límites posibles: primero profundidad 0, luego profundidad 1, luego profundidad 2, y así sucesivamente. En efecto, IDS combina las ventajas de DFS y BFS.

Búsqueda bidireccional

Búsqueda bidireccional Puede reducir enormemente la complejidad temporal, pero no siempre es aplicable. Sus requisitos de memoria pueden resultar poco prácticos.

El algoritmo BDS realiza una búsqueda simultánea tanto hacia adelante desde el estado inicial como hacia atrás desde el objetivo, y se detiene cuando ambas búsquedas se encuentran en el punto medio; sin embargo, este tipo de búsqueda no siempre es posible.

Costilla de primera calidad

Un número se denomina superprimo si es primo y todo número obtenido al eliminar cierta cantidad de dígitos del lado derecho de su expansión decimal es primo. Por ejemplo, 233 es un superprimo, porque 233, 23 y 2 son todos primos. Imprime una lista de todos los números superprimos de longitud n , para $n \leq 9$. El número 1 no es primo.

Para este problema, utilice la búsqueda en profundidad, ya que todas las respuestas estarán en el nivel n (el nivel más bajo) de la búsqueda.

La gira de Betsy

Un municipio cuadrado se ha dividido en n^2 parcelas cuadradas. La granja se encuentra en la parcela superior izquierda y el mercado en la inferior izquierda. Betsy recorre el municipio desde la granja hasta el mercado, pasando por cada parcela exactamente una vez. Escribe un programa que cuente cuántos recorridos únicos puede hacer Betsy desde la granja hasta el mercado para cualquier valor de $n \leq 6$.